IT314: Software Engineering
Lab Session: From Requirements to a Defensible System Design

Requirements Engineering / System Design

1. Lab objective

Apply early system-design concepts to a realistic problem. Turn an incomplete problem statement into measurable design goals, logical subsystems, interfaces, an architecture, state and failure models, and a revised design after a new constraint. Marks reward reasoning and consistency, not diagram quantity. There is no single correct architecture; decisions must fit your assumptions and assigned constraints.

2. Scenario: Ahmedabad weekend pickup sports

A small team plans an application for people in Ahmedabad who regularly play pickup games such as football, cricket or badminton at local turfs and courts. Hosts publish game details: sport, venue, time, skill level and the number of open player slots. Players search for compatible games, submit join requests, and receive responses. Users can skip a specific week without leaving their recurring group. The application supports cancellation, game completion, court-fee sharing, and notifications. The team has not decided how to divide the system, how components communicate, what load to support, or how failures are handled. Propose and defend a design.

3. Lab rules

- Work individually
- Record assumptions and distinguish them from facts. Each major design decision must include a reason and a consequence or trade-off.
- All diagrams and written explanations must agree. Keep dated design snapshots so changes can be traced.
- Disclose use of AI and be prepared to explain every submitted decision.
- Generic definitions or diagrams without scenario-specific reasoning will not receive full credit.

4. Tasks and required evidence

Task 1 — Clarify the problem

Prepare a one-page brief containing:

- Three questions you would ask the client before finalizing the design.
- At least five assumptions, explicitly labelled as assumptions.
- Four functional requirements and four quality requirements.
- Two features explicitly outside the first release.
- For each quality requirement, explain how it could be checked or measured. Avoid unmeasurable statements such as “the system should be fast.”

Task 2 — Set design goals and resolve a conflict

Choose four design goals, at least two measurable. For each, state why it matters, a target or evaluation method, and the evidence or metric that would test it. Identify a realistic conflict between two goals. Compare two possible resolutions, choose one, and explain what you give up. Reflect the chosen resolution in your architecture.

Task 3 — Decompose the system

Create a subsystem diagram and responsibility table with at least five logical subsystems. For each, describe its main responsibility, one responsibility it does not own, information consumed and produced, one dependency, and one failure or change it should handle without breaking unrelated parts. Do not create a separate subsystem for every class or screen. Explain why your decomposition fits your assigned profile.

Task 4 — Define interfaces and information flow

Specify at least four interfaces between subsystems. For each, state caller, receiver, purpose, inputs, outputs, and one error/failure result. Draw a sequence diagram or equivalent message-flow diagram for a player searching, submitting a request, the host responding, and both users being informed. Show which subsystem makes each decision and include an unsuccessful path.

Task 5 — Choose and justify an architecture

Select an approach for the first release (for example, layered architecture, modular monolith, or service-based design). Submit:

- An architecture/component diagram showing clients, backend components, storage, and external dependencies.
- A comparison of two plausible architecture options using at least three criteria.
- The selected option and the strongest reason for rejecting the alternative.
- A deployment sketch showing what runs together and what runs separately.
- An explanation of how the choice fits your design goals and assigned profile.

Do not assume each logical subsystem must be deployed as a separate service. If you choose multiple services, justify the operational cost and communication complexity.

Task 6 — Model a stateful object

Create a state machine for JoinRequest. Define states, events, conditions/guards, allowed transitions, and at least one terminal or effectively final state. Include two valid transitions and two invalid transitions, each with before-state, event, condition, expected result, and a rule that prevents an inconsistent outcome. Explain how the state model affects the interface or subsystem design.

Task 7 — Analyze failure and concurrency

Analyze both cases. For each, show event sequence, risk, proposed control, and what the user should see.

- Case A: Two players try to claim the final open slot at nearly the same time.
- Case B: A request is stored successfully, but the notification provider fails before the host receives the alert.
- Identify which component is authoritative, how duplicate/repeated actions are handled, and what happens if recovery is delayed. “Check it in the app” is not a sufficient control.

Task 8 — Respond to an instructor-issued change

After Tasks 1–7, there is a change request. Scenario: Support a second city without copying the whole codebase; replace the notification provider; allow a host to withdraw a published game; audit administrative actions; change matching to include skill-level compatibility; or allow a retry after a client timeout when the first request may have reached the server. Do not choose the change yourself. Submit a change-impact note identifying affected requirements, interfaces, subsystems, data, diagrams and tests; parts that should remain unchanged; one new risk; and a revised diagram or dated design snapshot.

Task 9 — Verify the design

Create six design-level test scenarios: at least two state-transition tests, one quality-goal test, one failure test, one concurrency/repeated-request test, and one test for the instructor-issued change. For each, specify setup, action, expected result, and the requirement or design decision being checked.

6. Submission package

- D1. Problem brief: questions, assumptions, scope, and requirements.
- D2. Design goals and trade-off decision.
- D3. Subsystem diagram and responsibility table.
- D4. Interface contracts and sequence/message-flow diagram.
- D5. Architecture and deployment diagrams, plus architecture comparison.
- D6. State machine and valid/invalid transition examples.
- D7. Failure and concurrency analysis.
- D8. Change-impact note and revised design snapshot.
- D9. Six design-level test scenarios.
- D10. One-page decision log: at least five decisions, alternatives considered, reasons, and consequences.

Submit one readable PDF or document. Include name, student ID, assigned profile, and date on the first page. Use consistent names across diagrams and tables.

7. Assessment rubric (100 marks)

Criterion	Marks	Evidence of strong work
Problem understanding, assumptions, requirements	10	Specific, relevant, internally consistent; missing information acknowledged.
Measurable goals and trade-off	12	Goals are testable; conflict and consequence are reasoned through.
Subsystem decomposition	15	Boundaries, responsibilities, and information ownership are clear.
Interfaces and interaction flow	12	Contracts are understandable; success and failure paths are represented.
Architecture and deployment reasoning	15	Alternatives are compared; choice fits goals, constraints, and operating capacity.

State model	10	States/events are distinct; valid and invalid transitions are consistent.
Failure, concurrency, recovery	10	Prevents inconsistent outcomes and defines user-visible behavior.
Change impact and verification	10	Impacts are traced; tests connect to requirements and decisions.
Decision log, clarity, oral defense	6	Student explains the work and acknowledges limitations.

8. Short oral defense

Be prepared to explain one randomly selected diagram or decision without reading a prepared script. The instructor may ask you to trace a request through the system, explain a rejected architecture option, show a failure path, identify an invalid transition, or apply a new constraint and point to the affected parts. You may refer to your diagrams; you must explain the reasoning behind them.

9. Responsible use of AI and references

Follow the instructor's stated policy. If AI tools are permitted, record the tool and task, distinguish suggestions from verified decisions, and validate any generated claims or diagrams. Generic output does not substitute for your assigned profile, design snapshots, trade-off analysis, change-impact trace, or oral defense. Do not submit measurements you cannot support or diagrams you cannot explain.

10. Student self-check

I. My assigned profile changes at least three design decisions.
II. My measurable goals have a way to be tested.
III. Subsystem responsibilities are consistent across the document.
IV. Interface contracts show inputs, outputs, and failure behavior.
V. My architecture comparison includes a real alternative and trade-offs.
VI. My state machine includes invalid transitions and expected behavior.
VII. I addressed both the last-slot race and notification failure.
VIII. My instructor-issued change resulted in a traceable revision.
IX. My test scenarios check stated requirements or design decisions.
X. I can explain every diagram and decision in my own words.

End of student assignment